

“锁”之一



锁与竞态

北京理工大学计算机学院
金旭亮

竞态 (race condition)



当多个线程同时访问一个共享资源时，如果不对它们读写的顺序作出规定，就有可能引发数据存取错误。这种情况被称为“**Race Condition** (竞态)”。



导致竞态的常见原因，是多个线程在没有采取任何控制措施的情况下并发地更新、读取同一个**共享**变量。



存取**局部**变量，不会导致竞态。

竞态的引发场景



引发竞态的两种场景：

read-modify-write（读-改-写）

check-then-act（检测而后行动）

关键在于——这些操作，都不是一次就可完成的。



竞态经常会导致的以下两种情况的发生：

- 脏读（Dirty Read）：线程读到了一个过时的数据。
- 丢失更新（Lost Update）：一个线程对数据所做的多次更新，其他线程“根本不知道有这回事”。

解决竞态的方法



解决方法是给要访问数据的线程添加一些规定，比如要求要访问共享的数据的线程，在访问前必须首先提出申请。



JVM会依据情况“批准”某个线程的申请并颁发“许可批文”，这一拿到“批文”的线程可以访问数据，其它要访问此数据的线程只能等待。

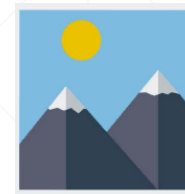


等当前拿到“批文”的线程完成对数据的访问之后，作废它的“批文”，JVM再给另一个线程颁发“访问许可”批文。



这个批文，就是“**锁 (lock)**”。锁可以理解为对共享数据进行保护的许可证。

互斥锁与读写锁

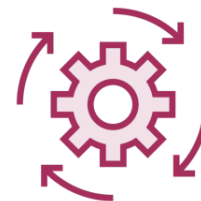


有些锁具有排他性 (Exclusive)，即一个锁一次只能被一个线程持有。因此，这种锁被称为“**互斥锁 (Mutex)**”。



另外还有一种允许多个线程同时读，单个线程互斥写的锁——**读写锁**，它可以被看作对“互斥锁”的一种改进，在“有读有写”的场景，可获得更高的性能。

自旋锁



有一种锁叫“**自旋锁 (SpinLock)**”，当一个线程在申请锁时，如果此锁已经被其他线程获取，那么该线程将会启动一个循环，不断地尝试获取锁，在这个过程中该线程一直保持运行状态，不会有上下文切换，因此也不会有因上下文切换而带来的性能损耗。

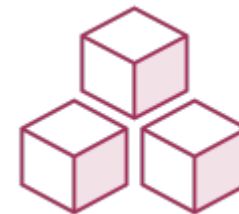


自旋锁 (SpinLock) 适合于每个线程都只占用锁很短的时间的场景，这种场景下，每个线程仅需“在线”等待一会就可以得到锁。



自旋锁的缺点是，如果获得锁的线程长时间占用而一直不释放，就会导致其它的等待线程一直在重试，“CPU空转”从而造成系统资源的浪费。

Java平台上的锁



1

内部锁 (Intrinsic Lock) : 通过synchronized关键字实现, Object类中的wait()和notify(), 也是基于内部锁实现的。

2

显式锁 (Explicit Lock) : 指java.concurrent.locks包中的Lock接口的实现类 (如ReentrantLock类) 。

与锁相关的几个概念



可重入性 (Reentrancy) : 一个线程在其持有一个锁的时候能否再次 (或者多次) 申请该锁。



锁的粒度 : 一个锁实例所保护的共享数据的数量大小, 称为该锁的“粒度 (Granularity)”。



锁泄漏 (Lock Leak) : 一个线程获得某个锁之后, 由于程序的错误、缺陷致使该锁一直无法被释放而导致其他线程一直无法获得该锁的现象。

锁的争用与调度



Java平台中锁的调度策略包括公平策略和非公平策略，相应的锁就被称为公平锁和非公平锁。

- 公平锁—排排坐，吃果果
- 非公平锁—允许插队，效率优先



内部锁属于非公平锁，而显式锁则既支持公平锁又支持非公平锁。



公平锁适合于锁被持有的时间相对长或者线程申请锁的平均间隔时间相对长的场景。

“线程安全”的概念



如果一个类在单线程环境下能够运作正常，并且在多线程环境下，其使用方不必对使用它的单线程代码做任何特殊的调整与修改，就能顺利运行并得到正确的结果，那么我们就称这个类是“**线程安全 (Thread-safe)**”的。



要编写“线程安全”的代码，基本思路如下：

- (1) 将并发访问转换为顺序串行访问
- (2) 在类或方法的内部，封装线程同步逻辑（比如使用锁）。



锁，是实现“线程安全”的主要技术手段。

“锁”为什么对“线程安全”至关重要？



锁可以通过**互斥**保障数据存取操作的**原子性**。



锁通过**同步读写缓存**保障了线程间数据存取操作的**可见性**。



锁利用上述两点（原子性和可见性），保障了线程数据操作的**有序性**。